

Ulagat — DevSecOps аудит и Defense-in-Depth (по состоянию на 01.07.2026)

Стек: Next.js 16 (App Router) на Vercel + Supabase (Postgres/GoTrue/Storage) + Stripe. ИИ в приложении не интегрирован (ментор из плана не реализован, флаги career/premium выключены). Документ грунтован на реальном коде репозитория, а не на общих советах.

Легенда статусов:

• [OK] —

Блок 0. Честная модель угроз под нашу архитектуру

Мы Serverless/BaaS. Классический «периметр сервера» (Nginx/Slowloris/TLS-конфиг) нам не принадлежит — им управляют Vercel и Supabase (см. блок 4). Реальная поверхность атаки:

1. БД напрямую через PostgREST (anon-ключ публичен) -> единственный рубеж это RLS. Это наш главный фронт.

2. Serve

Блок 1. Безопасность Supabase и защита данных (уровень БД)

1.1 RLS против IDOR/BOLA — наш главный рубеж [OK + PATCH-шаблон]

Единственная настоящая защита строк — RLS, потому что anon-ключ публичен и любой может дёргать PostgREST напрямую, минуя наш фронт и middleware. Проверено живыми атаками в этом проекте:

- profiles UPDATE имеет with_check, который пиннит role и external_skud_id к текущим значениям — самоэскалация роли и подмена СКУД на своей строке невозможны; qual (auth.uid() = id) режет чужие строки.

• substi

вставить

Канонический шаблон безопасной таблицы (копировать для новых таблиц):

```
alter table public.some_entity enable row level security;
```

-- ВАЖНО: ·

Правила, которые уже соблюдаем и должны соблюдать всегда:

- Po
permissiv
restrictive

Регулярная проверка (в CI или руками): таблиц без RLS быть не должно.

```
-- Должно вернуть 0 строк:
```

```
select reln
```

1.2 Adversarial-тест RLS (запускать после каждой миграции политик) [OK]

Мы используем безопасный приём: DO-блок с `set_config('request.jwt.claims', ...)` + `SET LOCAL role authenticated`, серия атак, и `RAISE EXCEPTION` в конце — транзакция откатывается, ничего не персистится. Пример проверенной батареи (ученик атакует profiles):

```
DO $$
```

```
DECLARE v_
```

Ожидаемо: self->admin BLOCKED, other->admin rows=0. Именно это мы и наблюдали.

1.3 Service Role Key vs Anon Key — где что можно [OK]

- `SUPABASE_SERVICE_ROLE_KEY` — обходит RLS. Живёт только в `lib/supabase/admin.ts` (`createAdminClient`), который импортируется исключительно из `Server Actions` / `Route Handlers` / `Server Components`. `autoRefreshToken:false`, `persistSession:false`. Никогда не в клиентском бандле, никогда не в `NEXT_PUBLIC_*`.

• Прави
(см. блок

1.4 Шифрование PII — честная инженерная оценка [PATCH, точно]

Что уже есть: Supabase шифрует данные at-rest на диске (AES-256) и in-transit (TLS). Пароли и e-mail хранятся в `auth.users` под управлением GoTrue, недоступны через наши таблицы. Наш PII в `profiles`: `full_name`, `grade`, `class_letter`, `avatar_url`, `external_skud_id`.

Честно: column-level шифрование через `pgcrypto` ломает `where`/индексы/`order by` и переносит управление ключом в приложение. Для имён и класса, которые нужны в фильтрах и `join`-ах (адресация замен по `grade+class_letter`), это вредно — потеряете RLS-фильтрацию и производительность. Поэтому шифруем точно только то, что (а) действительно чувствительно и (б) не участвует в фильтрах.

Кандидат №1 — `external_skud_id` (идентификатор пропуска/турникета). Паттерн через Supabase Vault (ключ в Vault, а не в коде):

```
create extension if not exists supabase_vault;      -- Vault для секретов/ключей
```

-- один раз

Для секретов приложения (Resend/Stripe/CRON) — они не в БД, а в Vercel Env (см. блок 2.3). Vault нужен только если хотите держать серверные ключи внутри Postgres.

Пост-квантовая криптография [N/A на уровне приложения]: гибридный обмен ключами (X25519MLKEM768) обеспечивает TLS-терминация Vercel/Cloudflare Edge — это уже дефолт на периметре в 2025–2026. На уровне нашего кода PQС внедрять нечего и не нужно; «шифровать PII пост-квантовыми алгоритмами в строках БД» — маркетинговый миф для нашего кейса.

Блок 2. Vercel / Edge — API и сеть

2.1 Rate Limiting на Edge (анти ИИ-брутфорс, анти-парсинг, анти Wallet-DDoS) [PATCH]

Сейчас rate-limit отсутствует — это реальный пробел (брутфорс логина/инвайт-кода, парсинг users/search). Так как у нас нет своего сервера, лимитируем в middleware.ts на Edge через распределённый счётчик. Нужен стор: Upstash Redis (@upstash/ratelimit) или Vercel KV.

```
npm i @upstash/ratelimit @upstash/redis
```

```
# Vercel en
```

lib/security/rate-limit.ts:

```
import { Ratelimit } from '@upstash/ratelimit';
```

```
import { R
```

Врезка в middleware.ts (в начало, до ролевых проверок):

```
import { limiters, clientKey } from '@lib/security/rate-limit';
```

Скоринг «подозрительности» (простая эвристика без ML): отсутствие ассерт-языка/нестандартный user-agent + всплеск 4xx с одного IP -> понижаем лимит на этот ключ (второй, более узкий limiter по rl:suspect:<ip>). ИИ-боты часто ходят без реалистичных заголовков.

2.2 Заголовки безопасности и CSP [ОК частично + PATCH]

Уже стоят в next.config.ts: Strict-Transport-Security (2 года, includeSubDomains, preload), X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, Referrer-Policy. Нет CSP — главный недостающий рубеж против XSS/кражи сессии.

Правильный способ для Next 16 App Router — nonce-based CSP через middleware (статический CSP ломает inline-скрипты Next). Готовый патч:

```
// в middleware.ts, формируем nonce и заголовки на каждый ответ
```

```
const nonce
```

Внедрять поэтапно: сначала content-security-policy-report-only + отчёты, убедиться, что ничего не сломалось (Stripe, Supabase realtime, картинки), затем переключить в enforce. Требует правки app/layout.tsx для проброса nonce в next/script. Не применяю автоматически — высокий риск сломать рендер, нужен прогон.

Дополнительно (низкий риск, применяю сразу — см. блок «Применённые патчи»): Permissions-Policy, X-Permitted-Cross-Domain-Policies, Cross-Origin-Resource-Policy.

2.3 CORS [N/A/OK]

Приложение same-origin: фронт дёргает свои же роуты того же домена. Отдельный CORS не нужен и не настроен — это правильно (нет Access-Control-Allow-Origin: *). Если появится внешний потребитель API, allowlist строго по Origin:

```
const ALLOW = new Set(['https://ulagat.vercel.app']);
```

```
const orig
```

Webhook Stripe (/api/webhooks/stripe) обязан проверять подпись stripe-signature (stripe.webhooks.constructEvent) — это не CORS, но это критичный API-рубеж; убедиться, что реализовано.

2.4 Env-переменные в Vercel [DASHBOARD]

- Секреты (SUPABASE_SERVICE_ROLE_KEY, STRIPE_SECRET_KEY, RESEND_API_KEY, CRON_SECRET, UPSTASH_*) — без префикса NEXT_PUBLIC_. Только NEXT_PUBLIC_SUPABASE_URL и NEXT_PUBLIC_SUPABASE_ANON_KEY публичны.

- Паз
Stripe/Re

2.5 «Периметр» (Nginx/TLS 1.3/Slowloris) [N/A — управляет Vercel]

TLS 1.3, отключение старых версий, защита от Slowloris/медленных атак и терминация PQC — это Vercel Edge (на базе Cloudflare). Своего reverse-proxy у нас нет, конфиг Nginx/Envoy писать некуда и не нужно. Что в нашей власти: HSTS+preload (есть), домен в hstspreload.org (<https://hstspreload.org>), включить в Vercel «Attack Challenge Mode» при инциденте.

Блок 3. Безопасность интегрированного ИИ [N/A сейчас + PATCH на будущее]

Честно: в приложении сейчас нет ни одного ИИ-эндпоинта (в package.json нет @ai-sdk/anthropic/openai; ментор из плана не реализован). Поэтому Prompt Injection/джейлбрейк/утечка системного промпта сейчас неприменимы — защищать нечего. Ниже — готовый guard-модуль на момент, когда ИИ появится, чтобы не внедрять его «на живую» без защиты.

```
// lib/ai/guard.ts — применять ДО и ПОСЛЕ вызова модели
```

```
const INJE
```

Ключевые правила будущего ИИ (Defense in Depth):

• Никог
user-mes

Блок 4. Аутентификация и сессии (Supabase Auth) [DASHBOARD + OK + PATCH]

Честно про пароли: хеширование паролей выполняет Supabase GoTrue (bcrypt), а не наш код. Мы не задаём параметры Argon2id (memory/parallelism/work factor) — это возможно только при self-host GoTrue, а мы на управляемом Supabase. Поэтому «внедрить Argon2id с точными параметрами» в

нашем кейсе — неверная постановка; правильные рычаги ниже.

- [DASHBOARD] Leaked Password Protection (Pwned Passwords / HaveIBeenPwned) — сейчас выключено (advisor auth_leaked_password_protection). Включить: Supabase -> Auth -> Passwords. Плюс min length >= 8/10.

- [DASH
(Supabas
сессии (3

```
// enroll
```

```
const { dat
```

Энфорс AAL2 для админ-зоны (в middleware.ts для /admin):

```
const { data: aal } = await supabase.auth.mfa.getAuthenticatorAssuranceLevel();
```

```
if (pathnam
```

MFA для персонала — самая рентабельная мера против захвата админ-аккаунта (после включённого leaked-password protection).

Блок 5. Цепочка поставок и мониторинг (DevSecOps)

5.1 SCA в CI/CD [PATCH — применяю сразу]

.github/dependabot.yml (авто-PR на уязвимые зависимости) + .github/workflows/security.yml (npm audit + запрет service-role в клиенте). Полные файлы — в применённых патчах ниже. Политика: PR блокируется при уязвимости уровня high/critical; Dependabot — еженедельно.

Дополнительно: npm ci с локфайлом (детерминированные сборки), периодически npm audit signatures (проверка подписей провенанса npm), не добавлять зависимости без проверки (кейс xlsx -> exceljs).

5.2 Структурные security-логи для SIEM/WAF [PATCH — применяю сразу]

Единый JSON-логгер критических событий без утечки PII (id вместо email, хеш вместо значения). Пример события «подозрительная смена email»:

```
{
```

```
"ts": "20
```

Никаких паролей, токенов, полных email, тел писем. Модуль-логгер — в применённых патчах ниже; вывод идёт в stdout (Vercel собирает в Log Drains -> можно направить в SIEM/Datadog/Better Stack).

5.3 Топ-10 чек-лист перед публичным релизом (Vercel + Supabase, лето 2026)

1. Supabase Advisors = 0 errors. Сейчас: 0 ошибок, ~108 warn. Разобрать варны: включить leaked-password protection, проверить public_bucket_allows_listing (публичные бакеты позволяют листинг файлов — ограничить, если там не только публичные картинки).

2. RLS
данными

Приложение: приоритеты внедрения

| Приоритет | Мера | Статус |

|---|---|--

RLS/BOLA-рубеш, разделение ключей, cron-секрет, HSTS/заголовки, ограничение саморегистрации роли — уже [ОК] и проверены живыми атаками.